

On the Construction of Reed-Solomon Codes

How do error correction codes recover missing data and how are those error correction codes constructed?

Mathematics

Word Count: 3904

Table of Contents

Chapter 1: The Beginnings of Data Recovery.....	2-4
Chapter 2: Reed-Solomon Code Properties.....	4-8
Chapter 3: Application of Properties.....	8-10
Chapter 4: Finite Galois Fields and Polynomials.....	10-16
Chapter 5: Original Construction.....	16-18

Chapter 1: The Beginnings of Data Recovery

Reed-Solomon codes offer a solution to a very fundamental problem in information theory and signal processing: how can data be sent such that if the data gets corrupted, can it be fully recovered? This paper aims to answer this question, and generalize it more broadly as: **How do error correction codes recover missing data and how are those error correction codes constructed?** Reed-Solomon codes, among other such codes, are one solution. The problem arises when there is noise within a *channel*. Data is prepared on the sending end, and sent over the channel. The message is received and then read. The data is encoded on the sending end, and when the receiver receives it, the entire message is decoded to get the original data back. See **Figure 1**.

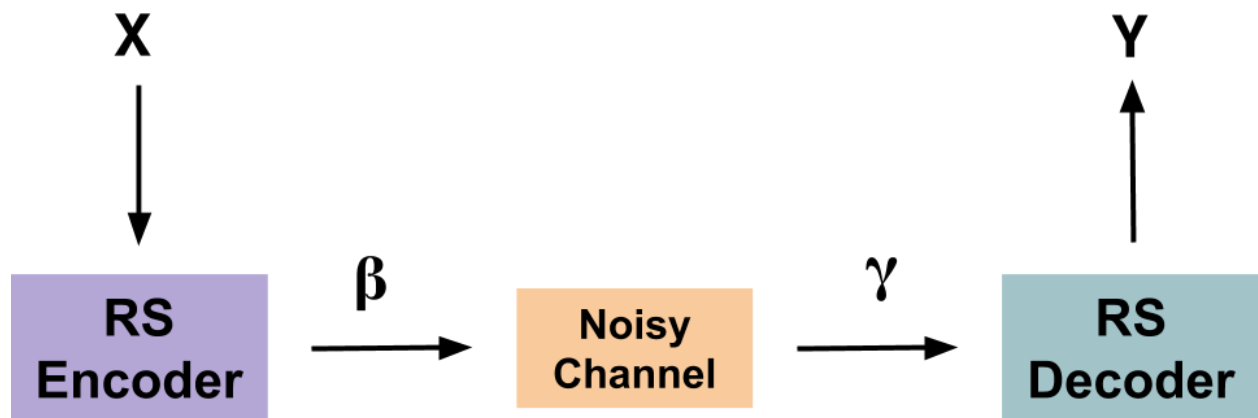


Figure 1: The Reed-Solomon Encoding/Decoding Process

When presented with the problem, the first attempted solution might be to repeat elements within the message. In an example message ROME, a solution might be to repeat each letter: RROOMMEE. If any one letter gets corrupted, the receiver can immediately know. Say,

the received message is HROOMMEE. The receiver is unsure if the message is ROME or HOME. The receiver concludes that an error in the first letter was made, as there is no pair of the same letter, but they are unsure of which word was meant, and therefore cannot *correct* the error, only *detect* the error. The next idea might be to repeat it twice for the balanced effect to disappear. For the data ROME, the sender would send RRROOOMMEEEE. Notice that any one block can be corrupted with one error, and still be correctable. RRR might become RHR, but the receiver could be more certain in the block being an R, rather than an H. Codes such as these are called repetition codes, and are useful for providing a basis on which information theory relies. However they are not very useful in the field. $\frac{1}{3}$ of the message consists of *information symbols*, and the other $\frac{2}{3}$ are considered *parity symbols*, a terrible rate for the amount of data that can be recovered vs the amount of data that was sent (Danziger, 2005). The problem of coming up with efficient codes to transmit data now becomes: **how does one append the least amount of *parity symbols* onto the end of the *information symbols* while retaining the ability to recover the data up to a predetermined tolerance.**

There are other issues with the use of repetition codes when used in practice. The repetition codes are too expensive to build in practice for a desired tolerance. Any repetition code can only correct $\frac{k-1}{2}$ errors, with k being the repetition number. In the example above, the repetition number was 3, and the code could correct 1 error. The goal is to make the codewords used as unique as possible, something difficult to achieve in practice, as all encoding is done using binary. In base 10, any error occurring in a symbol has 9 other options, however in binary, there is only one other option for a bit to be. The codewords being as unique as possible allows

for easy differentiation between codewords. This must then borne of structure. In our example, the structure was created through the use of repetition of the same symbol, that being R, O, M, or E. In practice, the structure of Reed-Solomon codes comes from using mathematical structures. Finite fields, also called Galois fields, represent the codewords of a code using polynomials, and manipulate them to guarantee the error tolerance using the properties of those polynomials.

For ease of explanation, numbers are used to represent information. The numbers are in binary, being either a 0 or a 1. A *bit* of a binary number is each 1 or 0, while the binary number is a *string* of consecutive of 1s and 0s. For example, 100110100 is a binary number, and the first two *bits* of that binary number are 10, and the last two are 00.

Chapter 2: Reed-Solomon Code Properties

Before discussion into the intricacies of how Reed-Solomon codes are constructed and used, a few terms must be cleared up. This whole subject, called coding theory, pioneered by Richard Hamming and Claude Shannon in the 1960's, coined a lot of the terms that are still used today (Hamming, 1950).

Sent information is divided up by the encoder to process in blocks called *messages*. The messages are k information symbols long. The *message* is then represented with k -length vectors. Each bit of the message has 2 possibilities, and there are k bits. Therefore, there are 2^k possible messages. The *encoder* takes each message and transforms it into an n -length vector, where $n = k + 2t$. Here, $2t$ is the number of parity symbols. Each transformed vector is called a

codeword. Each element within a codeword is a symbol. Symbols are both discrete and unique. The 2^k possible messages corresponds to 2^k possible codewords. The set of all possible codewords is called an (n, k) *block code*, or just the *code*.

The code's size becomes exponentially larger for large values of k . This was a key problem in terms of the applicability of Reed-Solomon codes in the 1950's when they were invented, as they were often too large to be able fit into memory in order to work with them. Decoding at the end of transmission involves looking through every possibility and choosing the codeword that is the most likely to be correct. Because there are 2^k possible codewords, this can be computationally inefficient with large values of k , and therefore more efficient algorithms are used today (Casey, 2000). The one used modernly is called the Berlekamp-Massey algorithm, pioneered by Elwyn Berlekamp and James Massey in 1969, and is outside of the scope of this paper (Casey, 2000).

To discuss the usefulness, tolerance, and other such properties of a Reed-Solomon code, there is terminology that must be defined. One such term is the *Hamming distance* between two codewords. The definition of the *Hamming distance*, $d = \{x, y \mid x, y \in \mathbb{Z}\}$ is the amount of data bits that need to be changed in order for one codeword to become the other. For example, $d(110, 010) = 1$, because only the bit in the 1st position needs to flip in order for the two vectors to equal each other. The order of the function does not matter, as for any x and y , $d(x, y) = d(y, x)$. Formally, this is written as $\sum_{i=1}^n |x_i - y_i|$. The goal of any code, as discussed

earlier, is to have as great a Hamming distance as possible. This helps differentiate the codewords from each other, thus making the decoding process easier.

The *minimum Hamming distance*, d_{min} is also important in discussing the effectiveness of a code. The *minimum Hamming distance* is defined as the lowest value of all of the *Hamming distances* between all codewords in a code. Using brute force, for a code size q , there are $q(q - 1)$ calculations that must be made. This is quadratic time and was too inefficient on the computers of the 1950's, especially if q is large. The d_{min} represents an upper bound for the number of errors that can occur in transmission that are still detectable. Logically, we can conclude that in any codeword, it is possible to detect $d_{min} - 1$ errors, since d_{min} errors in a codeword would result in either a different codeword, or nonsense that we are unable to decode. We are not able to correct $d_{min} - 1$ errors, just detect that number of them. Reed-Solomon codes are able to correct $t \leq \frac{d_{min}-1}{2}$ errors, as proven later in this paper.

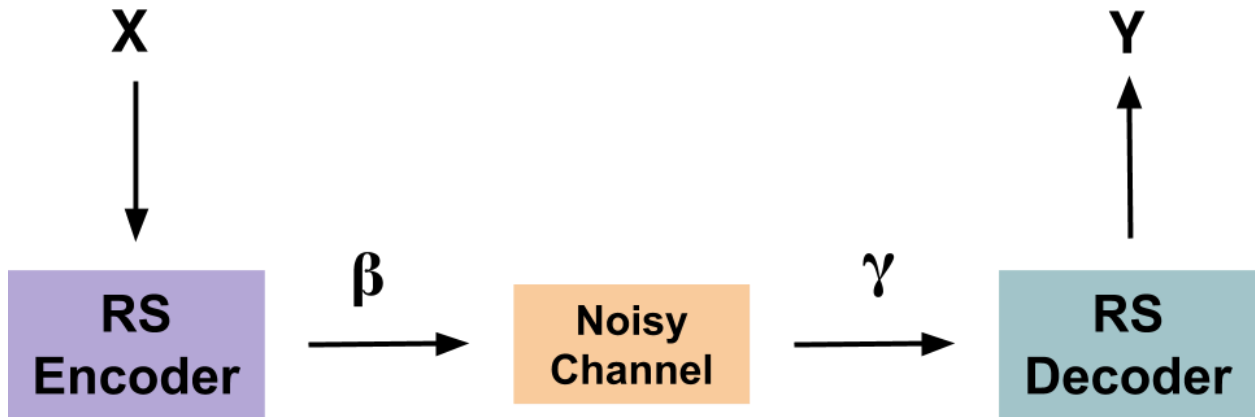


Figure 1: The Reed-Solomon Encoding/Decoding Process

With regards to **Figure 1**, if there are no errors, $X = Y$ and $\beta = \gamma$, and the decoding process is exactly the inverse of the encoding process.

Let C be the code for a message. From **Figure 1**, let $\beta \in C$ be the sent codeword and γ be the received codeword, with t of its symbols in error, so $t = d(\beta, \gamma)$. Defined earlier, there are n symbols in each codeword. Clearly, because t of the symbols are in error, $(n - t)$ symbols were transmitted correctly. The original codeword can be recovered if $(n - t) > d_{min}$. This can be seen by considering if there were d_{min} errors or more, γ could be Hamming-equidistant from any codeword in C , meaning any recovery attempt will have an equal (and random) chance of choosing the correct codeword, but also every other *incorrect* codeword.

Therefore, d_{min} must also be greater than $t + k - 1$, which can be expressed as $(t + k - 1) < d_{min}$. Since $(n - t) > d_{min}$ and $(t + k - 1) < d_{min}$, $(n - t)$ must be greater than $(t + k - 1)$, written as $(n - t) > (t + k - 1)$. This can be rearranged to solve for t . It can be seen $n > 2t + k - 1 \Rightarrow n - k \geq 2t \Rightarrow t \leq \frac{n-k}{2}$. Here, we arrive at an upper bound for the amount of errors any code can have, that is, half of the amount of parity symbols. As $n, k, t \in \mathbb{Z}$, the answer must also be an integer. As a symbol is either in error or not, the final t value is rounded down as a decimal. Aside, it can be shown that $t \leq \frac{d_{min}-1}{2}$, as well, the proof of which is similar to the proof above. These two expressions for t are sometimes called the *packing radius*, or the *error-correcting capability* of a code.

This is clearly a much better rate than even the most efficient of repetition codes. The best repetition codes used a small repetition rate, in order to keep the rate small. The smallest of which had a repetition rate of 3. Even then, the code rate for that code was only $\frac{1}{3}$, whereas this Reed-Solomon Code has a rate of $\frac{1}{2}$, and is infinitely scalable, whereas the code rate for the repetition code gets worse and worse the larger it gets.

Chapter 3: Application of Properties

Using the earlier example, we can see that the amount of errors our code can sustain and still be able to successfully decode back into the original information is

$t \leq \frac{n-k}{2} \Rightarrow t \leq \frac{12-5}{2} \Rightarrow t \leq 3.5$. As 3.5 errors are not possible (the symbol is either in error or not), the code can sustain up to three errors.

Let's restate the original example of sending the word ROME with the new terminology. We *encoded* the *information symbols* of $\{R, O, M, E\}$ into the codewords of $\{RRR, OOO, MMM, EEE\}$. To convert this into numbers to make things more familiar, notice we have 4 options for our information symbols. This corresponds to a length-2 binary vector for each. Note that both have 4 options. The information symbols are converted as follows in **Figure 2**:

R	00
O	01
M	10
E	11

Figure 2: The conversion from letters into length-2 binary vectors

Notice that in the original form, each letter was unique, and in the converted form, each length-2 binary vector is still unique. Therefore, this is a valid transformation. To transform the required codewords, we simply repeat the transformation. This can be seen in **Figure 3**.

RRR	000000
OOO	010101
MMM	101010
EEE	111111

Figure 3: Possible codewords in the repetition code example

This is our new code. Notice, again, our codewords are unique. In this case, $d = 3$, as the minimum distance is between RRR and OOO, and $d(RRR, OOO) = 3$. It only takes 3 bit shifts to change the codeword RRR into OOO. Therefore, $t = \frac{d-1}{2} = 1$. This aligns with what we calculated earlier using inspection, as well as with the letter example.

Note that this code is still needlessly inefficient. Many of the binary representations of the codeword triplets are unnecessarily long. Direct conversion from repetition code to Reed-Solomon code does not save efficiency, as repetition codes are built differently than Reed-Solomon codes.

Chapter 4: Finite Galois Fields and Polynomial Applications

To discuss the construction of Reed-Solomon codes, the structure needs to be addressed again. Up until this point, the symbols of the codewords that we have used are the binary symbols, that is, 0 and 1. These symbols do not necessarily have to be binary. They can be any symbols taken out of a finite field. Finite fields were pioneered by Évariste Galois, and are fundamental to coding theory and its applications (Cox, 2012). Because of their founder, they are also called Galois Fields, and are denoted by $GF(q)$, where q is the order, or size, of the field. All finite fields either have a prime subfield, or are a prime subfield. Prime subfields are finite fields with a *prime order*. This means that they are Galois Field $GF(p)$, where p is prime. Any field $GF(p^n)$ where $n > 1$ has the prime subfield $GF(p)$. This can be seen by if $n > 1$, the prime subfield IS the field, which means the field does not have its own, separate, prime subfield. Prime subfields are important to the understanding of finite fields, as they are used to represent the coefficients of various polynomials within the finite field.

There are three classes of polynomials that are important to the understanding, construction, and application of finite fields. They are called generator polynomials, primitive polynomials, and minimal polynomials.

Generator polynomials are special in that over any prime finite field $GF(p)$, each element α of the finite field is a root of the generator polynomial. It is called a generator simply because it is possible to construct the entire finite field knowing just generator polynomial and the size of the desired finite field. The generator also has cyclic properties, which is important for Reed-Solomon codes, as they are a type of code called *cyclic error-correcting codes*. The generator polynomial has another property that every codeword is *exactly* divisible by the generator polynomial when the codewords are represented in polynomial form. This is because codewords, when represented as polynomials, are in the form $c(x) = g(x)q(x) + r(x)$, with g being the generator polynomial.

The construction of the generator polynomial is simple. The roots of the generator polynomial must be the elements of the finite field that the code is working in, so the construction using the primitive element is simply:

$$\prod_{i=0}^{q-k-1} (x - \alpha^i)$$

The second type of polynomial that will become useful in the construction of Reed-Solomon codes are called primitive polynomials. All primitive polynomials are also irreducible polynomials, but not every irreducible polynomial can be primitive. In a finite field

$GF(q)$, where $q = p^k$, p is prime, and k is any positive integer, the primitive polynomial will have coefficients that are elements in $GF(p)$, where $GF(p)$ is the prime subfield of $GF(q)$. Primitive polynomials are defined by the fact and property that they are polynomials with coefficients in the *prime subfield* of any field $GF(q)$ in which a primitive element of $GF(q)$ is not a root. For fields that have multiple primitive elements, the primitive polynomial does not have a root of any of them. That is, if the set of primitive elements of a certain finite field is called H , the primitive polynomial $P(s) \neq 0$ for any $s \in H$. As an example, the primitive polynomials of the finite field $GF(8)$, or $GF(2^3)$, will have coefficients that are in $GF(2)$, that is, either a 1 or a 0. There is only one primitive polynomial in $GF(8)$. $GF(2)$ is also called the binary finite field, and polynomials over the binary finite field are called binary polynomials. This makes the primitive polynomials of any field with prime $p = 2$ binary polynomials. The primitive polynomial for $GF(8)$ is $p(x) = x^3 + x + 1$. This can be easily seen because none of the primitive elements of $GF(8)$ are roots of $x^3 + x + 1$. The binary field is especially useful as they closely resemble the architecture used within computers, and therefore the operations are very quick. The simple addition and multiplication tables for $GF(2)$ are shown below in **Figure 4**. Note that multiplication within the binary field closely resembles the binary operation of AND, and that addition resembles XOR.

Multiplication Table

*	0	1
0	0	0
1	0	1

Addition Table

+	0	1
0	0	1
1	1	0

Figure 4: Addition and multiplication tables for $GF(2)$

The third class of polynomials that will be useful to Reed-Solomon codes are called minimal polynomials. They are polynomials that are used to represent the members of any finite field. There is a unique polynomial for each element within a finite field, making this representation useful for arithmetic operations on elements within the field. The arithmetic operations are defined using one of the field's primitive polynomials, of which there may be many, but there is always guaranteed to be one. All arithmetic is defined using each elements' polynomial representation, and it is all performed modulo a primitive polynomial of the field. Performing the modulo operator on polynomials may be unfamiliar, but it is akin to factorization. For polynomial $P(x) = p(x)q(x) + c(x)$, the modulo between $P(x)$ and $q(x)$ is simply $c(x)$.

In arithmetic, the polynomial result is always reduced using the modulo of the primitive polynomial, and then the coefficients are reduced using an integer, the *prime order* of the field. This is why when adding $1 + 1$ in $GF(4)$ is 0, even though 2 is an element within $GF(4)$. For example, in $GF(8)$, let us add 6 and 7. The polynomial representations of those two elements

are $\alpha^2 + \alpha$ and $\alpha^2 + \alpha + 1$, respectively, with α being a primitive element of $GF(4)$. Adding them, the result is $2\alpha^2 + 2\alpha + 1$. The result of any addition will always have a degree less than that of the primitive polynomial, therefore, taking the result modulo the primitive polynomial is not necessary. This is analogous to asking for the remainder of dividing 8 by 13. It will always be 8 if the denominator is greater. Therefore, the result is taken modulo the *prime order*, which, for $GF(8)$, is 2. Therefore, $2\alpha^2 + 2\alpha + 1 \bmod (2) \Rightarrow 1$. Notice that the result of 1 is not the number 1, but the polynomial representation, which just so happens to equal the number 1. The addition of 4 and 7, for instance, returns $\alpha + 1$, which in $GF(8)$, equals the number 3. This is due to the *prime order* of the field being 2. This means that subtraction and addition are the same operation, and since the difference between 4 and 7 is 3, the answer is 3, even though the operation is addition.

The field has a primitive polynomial $p(x) = x^3 + x + 1$. By definition, the primitive polynomial must have the primitive element of the finite field as a root, so $p(\alpha) = 0$. The primitive polynomial can be rewritten in the form of a primitive identity of the finite field. Each primitive polynomial has its own primitive identity, of which there can be multiple in a finite field. The primitive identity of $p(x)$ is $0 = \alpha^3 + \alpha + 1$, or $\alpha^3 = \alpha + 1$.

Recalling that the primitive element is a generator element of the finite field, and raising it to consecutive powers generates every element within the finite field, the following table can be constructed using $p(x) = x^3 + x + 1$ as a primitive polynomial over $GF(8)$.

Exponential	Polynomial	Vector	Numerical
0	0	(0, 0, 0)	0
α^0	1	(0, 0, 1)	1
α^1	α	(0, 1, 0)	2
α^2	α^2	(1, 0, 0)	4
α^3	$\alpha + 1$	(0, 1, 1)	3
α^4	$\alpha^2 + \alpha$	(1, 1, 0)	6
α^5	$\alpha^2 + \alpha + 1$	(1, 1, 1)	7
α^6	$\alpha^2 + 1$	(1, 0, 1)	5

Table 1: Expansion of elements into polynomial, numerical, and vector representation in $GF(8)$

Each exponential representation is taken modulo the primitive polynomial. For a complex example, take α^6 , which represents the number 5. The process of simplifying from the exponential representation to the polynomial representation using the primitive identity is as follows.

$$\alpha^6 = \alpha^3 * \alpha^3 = (\alpha + 1)(\alpha + 1) = \alpha^2 + 2\alpha + 1 \pmod{2} = \alpha^2 + 1.$$

This same procedure of reducing modulo the primitive identity can be seen for every exponential representation of our primitive element.

The numerical representation in **Table 1** can be evaluated by using a known numerical primitive element, rather than the character α . For the finite field $GF(8)$, there is only one primitive element, which is the number 2. Other finite fields may have many, but every field is guaranteed to have at least one.

Chapter 5: Original Construction

In the original approach of Reed-Solomon codes, the construction was simple. Consider a k -length set of information symbols. They are called symbols because they are taken from the finite field $GF(q)$. Let us notate them with $\{m_0, m_1, m_2, \dots, m_{k-1}\}$. Then, consider a polynomial $P(x)$ such that $P(x) = m_0 + m_1x + m_2x^2 + \dots + m_{k-1}x^{k-1}$. Each codeword c in the code C is created by evaluating the polynomial using each element of the field $GF(q)$. Because each codeword c is a vector, the codeword is defined as $\{P(0), P(\alpha^0), P(\alpha^1), \dots, P(\alpha^{q-1})\}$. Then, all codewords in C are created by letting k cycle (hence the name cyclic code) through every possible value in $GF(q)$. In a binary field, that is, a field of the form $GF(2^k)$, there are only two values, so logically it follows that the code would have 2^k codewords. Because for each possibility of q , there are k symbols, a code generated from a k -length information block using symbols from a finite field $GF(q)$ will have q^k different codewords. The exponential for large values of q and k is why the computational complexity jumps as quickly as it does.

Another way of constructing the codewords of a code C is using the generator polynomial. The generator is, by definition, a divisor of every codeword within C , and each

codeword is simply the polynomial multiplication of the generator polynomial $g(x)$ and the k -length vector of information symbols. As an example, using the symbols from the prime binary field $GF(2)$, the information block is simply a set length binary vector. This length k defines the Reed-Solomon code (n, k) .

A Reed-Solomon code is a *cyclic code*, which means that shifting the coefficients of the polynomial of any codeword results in another codeword. This follows logically from the fact that all terms have every possible value of the information packet. For example, consider a codeword in a cyclic code represented by the binary vector 1101. Shifting each bit to the right and looping them, we arrive at 1110, which, by definition, must also be a codeword in the cyclic code. Cyclic codes are useful for data encoding because they are computationally more efficient to use than non-cyclic codes of the same size. This is because the Hamming distance between any two codes in a cyclic code is 1, but the Hamming distance is not guaranteed to be 1 or any constant in a non-cyclic code.

As Reed-Solomon codes are a type of error correction called *forward error correction*, meaning that before the data is sent, it is encoded. In **Figure 1**, this can be seen as transforming the data X into codewords β . This process transforms the k -length information block into codewords that the decoder can recognize. Not every data security scheme uses forward error correction, however. Methods such as retransmission send the data again to ensure the message has gotten through. Protocols such as HTTP for browsers use retransmission.

Reed-Solomon codes did not become widespread and usable until significant advances in their decoding was discovered. Originally, the decoders used exhaustive searches, looking for the closest codeword to the codeword that was received in error. This is very inefficient, as with large codes, it would take exponentially longer to complete. The key advance in the decoding was made with the Berlekamp-Massey Algorithm, developed in 1969 by Elwyn Berlekamp originally, and optimized by James Massey. It was a linear equation solver, and it improved on the exponential nature of the exhaustive searches used originally.

Anantharam, V., Raghavan, A., Pakzad, P., Ibanez, M., Spring 2005: Error Control Coding. Accessed May 12 2019. <https://people.eecs.berkeley.edu/~ananth/229BSpr05/ffnotes05.pdf>

Belk, J.: Minimal Polynomials. Accessed May 13th 2019, <http://faculty.bard.edu/~belk/math318/MinimalPolynomials.pdf>.

Casey, E., 2000: Berlekamp-Massey Algorithm. Accessed 12 May 2019. REU Report, University of Minnesota, 10pp. http://www-users.math.umn.edu/~garrett/students/reu/MB_algorithm.pdf.

Cherowitzo, B.: Finite Fields. Accessed 16 May 2019. <http://math.ucdenver.edu/~wcherowi/courses/m6406/finflds.pdf>

Conrad, K.: Finite Fields. Accessed 16 May 2019. 16pp. <https://kconrad.math.uconn.edu/blurbs/galoistheory/finitefields.pdf>.

Cox, R. 2012: Finite Field Arithmetic and Reed-Solomon Coding. Accessed 12 May 2019. <https://research.swtch.com/field>.

Cox, R., 2012: QArt Codes. Accessed 12 May 2019, <https://research.swtch.com/qart>.

Fiedler, J. 2004: Hamming Codes.

<https://orion.math.iastate.edu/linglong/Math690F04/HammingCodes.pdf>

Forney, G. D.: Reed-Solomon Codes. Accessed 13 May 2019,

<https://web.stanford.edu/class/ee392d/Chap8.pdf>.

Danziger, P., 2005: Linear Codes. Accessed 12 May 2019.

<https://math.ryerson.ca/~danziger/professor/MTH108/Handouts/codes.pdf>

David Wheeler (<https://math.stackexchange.com/users/23285/david-wheeler>), Compute the addition table of the finite field $\mathbb{F}_8$, URL (version: 2017-12-19):

<https://math.stackexchange.com/q/2572635>

Dilip Sarwate (<https://math.stackexchange.com/users/15941/dilip-sarwate>), Why are the roots of generator polynomials consecutive powers?, URL (version: 2017-05-02):

<https://math.stackexchange.com/q/2263041>

Dong, C: Math in Network Security: A Crash Course.

<https://www.doc.ic.ac.uk/~mrh/330tutor/ch04s04.html>

Hamming, R., 1950: Error Detecting and Error Correcting Codes. The Bell System Technical Journal, 29, 1-14, <https://signallake.com/innovation/hamming.pdf>.

Han, Y. S.: Introduction to Finite Fields. Accessed 16 May 2019.

<http://web.ntpu.edu.tw/~yshan/algebra.pdf>

Ho, S. Y., 2014: Polynomial Codes. Accessed 12 May 2019,

math.mit.edu/~shor/18.310/polycode.pdf.

Hu, J.: Reed Solomon Codes. Accessed 18 May 2019.

<https://homes.cs.washington.edu/~jrl/teaching/cseP531sp16/presentations/pres4.pdf>.

Lin, S. Costello Jr, D. J., 1983: Error Control Coding: Fundamentals and Applications. Prentice Hall, 551pp.

<http://index-of.co.uk/Information-Theory/Error%20Control%20Coding%20Fundamentals%20and%20Applications%20-%20Shu%20Lin.pdf>

Neunhoefffer, M.: Finite Fields. Accessed 12 May 2019,

<http://www.math.rwth-aachen.de/~Max.Neunhoefffer/Teaching/ff/ffchap3.pdf>.

Pollara, F., 1985: A Software Simulation Study of a (255,223) Reed-Solomon Encoder/Decoder.

JPL., 85-23, 1-14, <https://ntrs.nasa.gov/archive/nasa/casi.ntrs.nasa.gov/19850019880.pdf>

Riley, M., Richardson, I, 1998: Reed-Solomon Codes. Accessed 16 May 2019.

https://www.cs.cmu.edu/~guyb/realworld/reedsolomon/reed_solomon_codes.html

Rosenberg, B., 2003: Example of GF(9). Accessed 12 May 2019,

<http://www.cs.miami.edu/home/burt/learning/Csc609.032/notes/gf9example.html>.

Rosenthal, J., Smarandache, R.,: Maximum Distance Convolutional Codes. Accessed 12 May

2019. <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.52.2658&rep=rep1&type=pdf>

Rudra, A., 2007: Error Correcting Codes: Combinatorics, Algorithms, and Applications.

Accessed 12 May 2019.

<https://cse.buffalo.edu/faculty/atri/courses/coding-theory/lectures/lect12.pdf>.

Snowball (<https://math.stackexchange.com/users/24875/snowball>), Primitive elements of GF(8),

URL (version: 2016-07-10): <https://math.stackexchange.com/q/988262>.

Tervo, R., 2010: GF(2) Calculator. Accessed 16 May 2019.

<http://www.ee.unb.ca/cgi-bin/tervo/calc.pl?num=1101&den=1011&f=a&e=1&p=1&m=1>

The Field GF(8). Accessed 17 May 2019.

<http://homepages.math.uic.edu/~leon/mcs425-s08/handouts/field.pdf>

Tomasz (<https://math.stackexchange.com/users/30222/tomasz>), Galois Field $GF(4)$, URL (version: 2012-07-19): <https://math.stackexchange.com/q/172995>.

UCDenver: Introduction to Finite Fields. Accessed May 12 2019, <http://math.ucdenver.edu/~wcherowi/courses/m6406/csln4.html>.

Weisstein, Eric W. "Finite Field." From MathWorld--A Wolfram Web Resource. <http://mathworld.wolfram.com/FiniteField.html>

Wicker, S. B., Bhargava, V. K.: Polynomial codes over certain finite fields. An Introduction to Finite Fields. 1-16. <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.93.8022&rep=rep1&type=pdf>.

Wolf, J.K., Evans, P., Pfister, H. D., 2006: An Introduction to Reed-Solomon Codes. Accessed 13 May 2019, <http://pfister.ee.duke.edu/courses/ecen604/rspoly.pdf>